This chapter covers I/O (Input/Output) systems, including how peripheral devices communicate with the CPU and memory, and how this communication is managed using:

- Buses
- Interrupts
- DMA (Direct Memory Access)
- I/O Channels

🔹 **Chapter 8: Input/Output – Interrupts and DMA**

📘 **Overview**

What Will Be Covered:

1. How I/O devices connect to the system
2. Communication via buses
3. Handling I/O using:
    - Programmed I/O
    - Interrupt-driven I/O
    - DMA (Direct Memory Access)
4. Evolution of I/O techniques
5. Advanced I/O methods like I/O channels

🔹 **Slide / Section: Introduction to I/O Systems**

📌 **Summary:**

- I/O devices vary widely (keyboard, disk, network, etc.)
- Need efficient ways to transfer data between the CPU/memory and these devices.
- Goal: Make I/O fast, flexible, and not waste CPU time.

🔹 **Slide / Section: Buses – Connecting I/O Devices**

📌 **Summary:**

A bus is a shared communication link connecting multiple components (CPU, memory, I/O devices).

🧠 **Key Concepts:**

Type	Description
Control lines	Signals like read/write enable communication
Data lines	Carry actual data
Address lines	Specify which device or memory location is being accessed

⚙️ **Bus Transaction Steps:**

1. Send address (to select target)
2. Transfer data (read or write)

🎯 **Example:**

```
lw $t0, 0($a0)   # Read from memory
sw $t1, 4($a0)   # Write to memory
```

→ These operations use the bus to access memory.

🤔 **Student Question:**

"Why is a bus considered a bottleneck?"
➤ Because only one transaction can happen at a time — it's a shared resource.

🔹 **Slide / Section: Types of Buses**

📌 **Summary:**

There are several types of buses used in computer architecture:

Bus Type	Characteristics
Processor-memory bus	Short, high-speed, connects CPU and memory
I/O bus	Longer, connects many devices, variable speed
Backplane bus	Connects CPU, memory, and I/O on one board

🧠 **Teaching Tip:**

Use real-world examples:

- USB port = I/O bus
- RAM connected directly to CPU = processor-memory bus

🔹 **Slide / Section: Synchronous vs Asynchronous Buses**

📌 **Summary:**

Type	Clocking	Speed	Complexity
Synchronous	Uses a clock signal	Faster	Less flexible
Asynchronous	Handshaking protocol	Slower but more reliable	More complex

🔄 **Handshaking Example:**

- The device says "ready."The
- CPU reads/writes data
- Device says "done."

🎯 **Student Question:**

“When would asynchronous communication be better than synchronous?”

🔹 **Slide / Section: Interrupts**

📌 **Summary:**

An interrupt allows an I/O device to signal the CPU that it needs attention.

🧠 **How It Works:**

1. The CPU is executing instructions normally.
2. A device raises an interrupt signal.
3. The CPU stops the current task and jumps to an interrupt service routine (ISR).
4. After handling the interrupt, the CPU resumes normal execution.

🔄 **Real-Life Analogy:**

Think of an interrupt like a doorbell: someone is waiting outside, stop what you're doing and respond.

📋 **Advantages of Interrupts:**

- The CPU doesn't have to wait for slow devices
- Supports multitasking and responsive systems

🎯 **Student Question:**

“What would happen if the CPU had to poll every device continuously?”

🔹 **Slide / Section: Instruction Cycle with Interrupt**

📌 **Summary:**

The basic instruction cycle expands to include interrupts:

[Fetch] → [Decode] → [Execute] → [Check for Interrupt]

If an interrupt is pending:

- Save the current state
- Jump to the interrupt handler
- Resume after handling

🔹 **Slide / Section: DMA (Direct Memory Access)**

📌 **Summary:**

DMA allows I/O devices to transfer data directly to/from memory without involving the CPU.

✅ **Benefits:**

- Frees up the CPU
- High throughput for large data transfers
- Efficient for disk/network transfers

🔄 **How It Works:**

1. The CPU sets up the transfer (source, destination, size).
2. The DMA controller takes over the bus.
3. Data moves directly between the device and memory.
4. When done, DMA signals the CPU via an interrupt.

🧩 **Example:**

Copying a large file from disk to memory:

- Without DMA: CPU copies each word → very slow
- With DMA, the CPU initiates the transfer and continues other work

🔹 **Slide / Section: DMA Configurations**

📌 **Summary:**

There are different ways to implement DMA:

Configuration	Description
Single bus, detached DMA controller	Simple design, shares the main bus with the CPU
Integrated into the I/O module	Dedicated DMA per device
Separate I/O bus	Reduces contention on the main bus

🧠 **Design Trade-off:**

- More dedicated hardware = higher performance but higher cost

🔹 **Slide / Section: Evolution of I/O Methods**

📌 **Summary:**

I/O has evolved through five stages:

Stage	Description
1. The processor controls the device directly	No abstraction — inefficient

2. Programmed I/O	The CPU waits for device readiness
3. Interrupt-driven I/O	The CPU is notified when ready
4. DMA	CPU offloads bulk transfers
5. I/O Channels	Smart I/O processors run programs independently

🧠 **Why It Matters:**

Each stage improves efficiency by reducing CPU involvement.

🎯 **Student Question:**

“How does an I/O channel differ from a DMA controller?”

🔹 **Slide / Section: I/O Channels**

📌 **Summary:**

An I/O channel is a specialized processor that:

- Fetches and executes I/O commands from memory
- Transfers data autonomously
- Reports completion via interruption

🧩 **Example:**

Instead of the CPU copying 1MB of data from disk:

- The CPU tells the I/O channel: “Copy 1MB from disk to address X”
- I/O channel does it without further CPU help

📈 **Advantages:**

- Offloads complex I/O tasks
- Can handle multiple transfers
- Increases system throughput

🎯 **Student Question:**

"Is an I/O channel a form of multiprocessing? Why or why not?"

🔹 **Slide / Section: Selector vs Multiplexer Channels**

📌 **Summary:**

Two types of I/O channels:

Channel Type	Function
Selector Channel	Handles one device at a time (like a single-threaded program)
Multiplexor Channel	Supports Time Division Multiplexing (TDM)— handles multiple devices simultaneously

🔄 **Comparison:**

- Selector: Fast for one device, poor concurrency
- Multiplexor: Slower per device, great for many devices

🎯 **Teaching Tip:**

Use a restaurant analogy:

- Selector = One waiter serving one table at a time
- Multiplexor = One waiter serving multiple tables in turns

✅ **Summary Table: I/O Techniques Compared**

Technique	CPU Involvement	Throughput	Best For
Programmed I/O	High	Low	Small transfers
Interrupt-driven I/O	Moderate	Moderate	Responsive systems
DMA	Low	High	Large data transfers
I/O Channels	Very Low	Very High	Complex I/O systems